

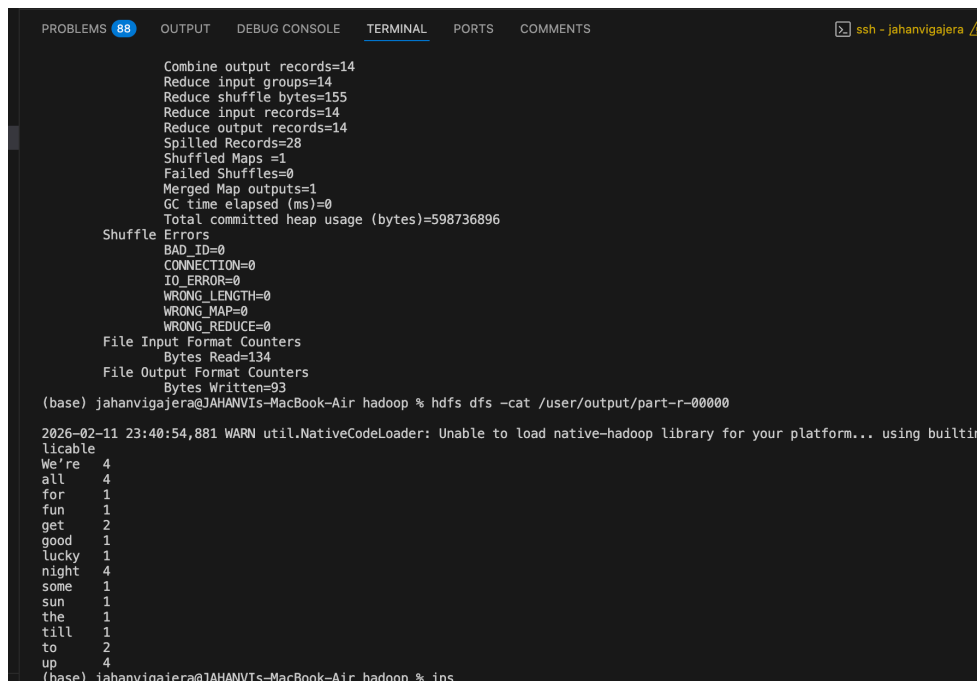
Assignment 1

Course: ML with BigData

Jahanvi Gajera (M25CSA012)

Part A: HADOOP TASK

Question 1:



The screenshot shows a Hadoop terminal window with the following content:

```
PROBLEMS 88 OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS ssh - jahanvigajera /

Combine output records=14
Reduce input groups=14
Reduce shuffle bytes=155
Reduce input records=14
Reduce output records=14
Spilled Records=28
Shuffled Maps =1
Failed Shuffles=0
Merged Map outputs=1
GC time elapsed (ms)=0
Total committed heap usage (bytes)=598736896

Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

File Input Format Counters
Bytes Read=134
File Output Format Counters
Bytes Written=93

(base) jahanvigajera@JAHANVIS-MacBook-Air hadoop % hdfs dfs -cat /user/output/part-r-00000

2026-02-11 23:40:54,881 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin
licable
We're 4
all 4
for 1
fun 1
get 2
good 1
lucky 1
night 4
some 1
sun 1
the 1
till 1
to 2
up 4
(base) jahanvigajera@JAHANVIS-MacBook-Air hadoop % jps
```

Question 2:

The input pairs for the Map phase will be the following:

- (0, "We're up all night to the sun")
- (31, "We're up all night to get some")
- (63, "We're up all night for good fun")
- (95, "We're up all night to get lucky")

1. What will the output pairs look like?

➔ For line 1:

("We're", 1) ("up", 1) ("all", 1) ("night", 1) ("to", 1) ("the", 1) ("sun", 1)

➔ For line 2:

("We're", 1) ("up", 1) ("all", 1) ("night", 1) ("to", 1) ("get", 1) ("some", 1)

➔ For line 3:

("We're", 1) ("up", 1) ("all", 1) ("night", 1) ("for", 1) ("good", 1) ("fun", 1)

➔ For line 4:

("We're", 1) ("up", 1) ("all", 1) ("night", 1) ("to", 1) ("get", 1) ("lucky", 1)

2. What will be the types of keys and values of the input and output pairs in the Map phase?

➔ Input pair type

- Key (byte offset of the line) – LongWritable
- Value (line of text) – Text

➔ Output pair type

- Key (word from the line) – Text
- Value (count 1 for each word) – IntWritable

Question 3:

For the Reduce phase, some of the output pairs will be the following:

("up", 4) ("to", 3) ("get", 2) ("lucky", 1)

1. What will the input pairs look like?

➔ ("up", [1, 1, 1, 1])

("to", [1, 1, 1])

("get", [1, 1])

("lucky", [1])

2. What will be the types of keys and values of the input and output pairs in the Reduce phase?

➔ Input pair type

- a. Key (byte offset of the line) – Text
- b. Value (line of text) – Text

➔ Output pair type

- c. Key (word from the line) – Text
- d. Value (count 1 for each word) – IntWritable

Question 4:

```
public static class Map
extends Mapper<LongWritable, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
```

```
public static class Reduce
    extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();
```

Question 5:

```
public void map(LongWritable key, Text value, Context context)
    throws IOException, InterruptedException {

    String line = value.toString().replaceAll("[^a-zA-Z ]", "").toLowerCase();
    StringTokenizer tokenizer = new StringTokenizer(line);

    while (tokenizer.hasMoreTokens()) {
        word.set(tokenizer.nextToken());
        context.write(word, one);
    }
}
```

Question 6:

```
public void reduce(Text key, Iterable<IntWritable> values, Context context)
    throws IOException, InterruptedException {

    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    context.write(key, new IntWritable(sum));
}
```

Question 7:

→ (base) jahanvigajera@JAHANVIS-MacBook-Air MLB % hdfs dfs -copyFromLocal
/Users/jahanvigajera/Desktop/Assignment/MLB
/hadoop_task/D184MB/200.txt /user/input/

```
● (base) jahanvigajera@JAHANVIS-MacBook-Air MLB % hdfs dfs -ls /user/input
2026-02-14 11:24:45,354 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your plat
Found 3 items
-rw-r--r--  1 jahanvigajera supergroup    8312639 2026-02-12 00:33 /user/input/200.txt
drwxr-xr-x  - jahanvigajera supergroup      0 2026-02-13 00:10 /user/input/Gutenberg
-rw-r--r--  1 jahanvigajera supergroup     134 2026-02-11 23:39 /user/input/input.txt
```

→ First compile WordCount.java () and get WordCount.jar

- (base) jahanvigajera@JAHANVIS-MacBook-Air MLB % javac -
classpath `hadoop classpath` -d .

/Users/jahanvigajera/Desktop/Assignment/MLB/hadoop_task/WordCount.java

- (base) jahanvigajera@JAHANVIs-MacBook-Air MLB % jar -cvf wordcount.jar *.class

```
● (base) jahanvigajera@JAHANVIs-MacBook-Air MLB % jar -cvf wordcount.jar *.class
added manifest
adding: WordCount$Map.class(in = 2009) (out= 876)(deflated 56%)
adding: WordCount$Reduce.class(in = 1727) (out= 742)(deflated 57%)
adding: WordCount.class(in = 2247) (out= 1205)(deflated 46%)
```

➔ (base) jahanvigajera@JAHANVIs-MacBook-Air MLB % hadoop jar

/Users/jahanvigajera/Desktop/Assignment/MLB/hadoop_task/wordcount.jar
WordCount /user/input/200.txt /user/output1234

```
● (base) jahanvigajera@JAHANVIs-MacBook-Air MLB % hadoop jar /Users/jahanvigajera/Desktop/Assignment/MLB/hadoop_task/wordcount.jar WordCount /user/input/200.txt /user/output1234
2026-02-14 11:27:58,596 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-02-14 11:27:59,786 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-02-14 11:27:59,833 WARN mapreduce.JobResourceUploader: No job jar file set. User classes may not be found. See Job or Job#setJar(String).
2026-02-14 11:27:59,888 INFO input.FileInputFormat: Total input files to process : 1
2026-02-14 11:28:00,161 INFO mapreduce.JobSubmitter: number of splits:1
2026-02-14 11:28:00,412 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local1866320914_0001
2026-02-14 11:28:00,414 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-02-14 11:28:00,612 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-02-14 11:28:00,614 INFO mapreduce.Job: Running job: job_local1866320914_0001
2026-02-14 11:28:00,615 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2026-02-14 11:28:00,620 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-02-14 11:28:00,622 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-02-14 11:28:00,622 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-02-14 11:28:00,624 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
2026-02-14 11:28:00,712 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-02-14 11:28:00,713 INFO mapred.LocalJobRunner: Starting task: attempt_local1866320914_0001_m_000000_0
2026-02-14 11:28:00,730 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-02-14 11:28:00,730 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-02-14 11:28:00,730 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-02-14 11:28:00,737 INFO util.ProcfsBasedProcessTree: ProcfsBasedProcessTree currently is supported only on Linux.
2026-02-14 11:28:00,737 INFO mapred.Task: Using ResourceCalculatorProcessTree : null
2026-02-14 11:28:00,744 INFO mapred.MapTask: Processing split: hdfs://127.0.0.1:9000/user/input/200.txt:0+8312639
2026-02-14 11:28:00,786 INFO mapred.MapTask: (EQUATOR) 0 kvi 26214396(104857584)
2026-02-14 11:28:00,786 INFO mapred.MapTask: mapreduce.task.io.sort.mb: 100
2026-02-14 11:28:00,786 INFO mapred.MapTask: soft limit at 83886080
```

➔ Output:

```
1 a 25344
2 aa 14
3 aaa 1
4 aaaff 1
5 aabborgt 1
6 aachen 7
7 aachens 1
8 aad 1
9 aadms 1
```

Question 8:

```
● (base) jahanvigajera@JAHANVIs-MacBook-Air MLB % hdfs dfs -ls /user/input
2026-02-14 11:24:45,354 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform
Found 3 items
-rw-r--r-- 1 jahanvigajera supergroup 8312639 2026-02-12 00:33 /user/input/200.txt
drwxr-xr-x - jahanvigajera supergroup 0 2026-02-13 00:10 /user/input/Gutenberg
-rw-r--r-- 1 jahanvigajera supergroup 134 2026-02-11 23:39 /user/input/input.txt
```

➔ Here I set replication factor 1.

Question 9:

```
wordcount.log x WordCount.class WordCount.java 9+
wordcount.log
1 Total Execution Time for split size (32): 4334 milliseconds
2 Total Execution Time for split size (64): 4315 milliseconds
3 Total Execution Time for split size (128): 4307 milliseconds
```

→ Execution time **slightly decreased** as the split size increased. Here dataset is very small, so that point doesn't affect heavily. For **large datasets on multi-node clusters**, smaller splits may improve parallelism but can increase overhead.

→ Optimal split size usually depends on:

- HDFS block size
- Cluster size
- Data volume

Part B: SPARK TASK

Question 10: Book Metadata Extraction and Analysis

```
>>> books_df.select("title", "language", "encoding").show(5, truncate=False)
...
+-----+-----+-----+
|title|language|encoding|
+-----+-----+-----+
|THE TIGER OF MYSORE:\r|English|UTF-8|
|SENSE AND SENSIBILITY\r|English|UTF-8|
|Bonnie Prince Charlie\r|English|UTF-8|
|DIDEROT AND THE ENCYCLOPÆDISTS\r|English|UTF-8|
|A SMALL BOY AND OTHERS\r|English|UTF-8|
+-----+-----+-----+
only showing top 5 rows
```

```
>>> books_df.select("release_year").distinct().show(20)
...
+-----+
|release_year|
+-----+
|1848|
|1811|
|1905|
|1895|
|1682|
|1854|
|1728|
|1684|
|1923|
|1912|
|1870|
|1886|
|1902|
|1914|
|1918|
```

```

• >>> from pyspark.sql import functions as F
• >>> books_df = books_df.withColumn(
...     "title",
...     F.trim(
...         F.split("value", "\n").getItem(1)
...     )
... )
• >>> books_df.select("title").show(10, truncate=False)

```

```

+-----+
|title|
+-----+
|OAK OPENINGS\r|
|\r|
|THE TIGER OF MYSORE\r|
|PRIDE AND PREJUDICE\r|
|SENSE AND SENSIBILITY\r|
|THE REFUGEES\r|
|Bonnie Prince Charlie\r|
|POETICAL WORKS\r|
|DIDEROT AND THE ENCYCLOPÆDISTS\r|
|A SMALL BOY AND OTHERS\r|
+-----+

```

only showing top 10 rows

```

• >>> books_df.select(
...     F.split("value", "\n").getItem(3)
... ).show(5, truncate=False)

```

```

+-----+
|split(value, \n, -1)[3]|
+-----+
|JAMES FENNIMORE COOPER\r|
|\r|
|A Story of the War with Tippoo Saib\r|
|By Jane Austen\r|
|by Jane Austen\r|
+-----+

```

only showing top 5 rows

```

• >>> books_df = books_df.withColumn(
...     "author",
...     F.trim(
...         F.split("value", "\n").getItem(3)
...     )
... )
• >>> books_df.select(
...     F.avg(F.length("title"))
... ).show()

```

```

+-----+
|avg(length(title))|
+-----+
| 18.78048780487805|
+-----+

```

Question 11: TF-IDF and Book Similarity

```

• >>> print(similarity_matrix)
[[1. 0.02929357 0.08019644 ... 0.02306629 0.05416131 0.03435538]
 [0.02929357 1. 0.02838058 ... 0.00846875 0.02381619 0.01196021]
 [0.08019644 0.02838058 1. ... 0.0241486 0.03788141 0.0320787 ]
 ...
 [0.02306629 0.00846875 0.0241486 ... 1. 0.01909943 0.01446463]
 [0.05416131 0.02381619 0.03788141 ... 0.01909943 1. 0.02042435]
 [0.03435538 0.01196021 0.0320787 ... 0.01446463 0.02042435 1. ]]

```

1. Explain TF and IDF. Why is TF-IDF useful for weighting the importance of words in a document?

→ **Term Frequency (TF)** measures how often a word appears in a document.

$$TF(t, d) = \frac{\text{Total number of terms in document } d}{\text{Number of times term } t \text{ appears in document } d}$$

If a word appears many times in a document, its TF is high. It captures **local importance**

→ **Inverse Document Frequency (IDF)** measures how rare or common a word is across all documents.

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

Where:

- N = total number of documents
- $df(t)$ = number of documents containing the term t

If a word appears in many documents, then IDF is low. If a word appears in a few documents then DF is high.

→ **TF-IDF**

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

It combines:

- Local importance (TF)
- Global uniqueness (IDF)

It highlights words that are important in one document, but not common across all documents.

2. How is cosine similarity calculated? Why is it appropriate to compare documents represented as TF-IDF vectors?

→ Cosine similarity measures the similarity between two vectors.

For two vectors **A** and **B**:

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \|B\|}$$

Where:

- $A \cdot B$ = dot product
- $\|A\|$ = magnitude of vector A
- $\|B\|$ = magnitude of vector B

Result range:

- **1** → identical direction (very similar)
- **0** → no similarity
- **-1** → opposite direction (rare in TF-IDF since values ≥ 0)

→ When documents are represented as TF-IDF vectors:

- Each dimension = a word
- Each value = TF-IDF weight

➔ It is appropriate because it:

- Normalizes document length
- Works well with sparse high-dimensional data
- Captures similarity in word importance patterns

Conclusion: TF-IDF determines its coordinates. Cosine similarity checks how close their directions are.

- I had generated a CSV file for the cosine similarity metric.

Question 12: Author Influence Network

```
>>> books_df.select("author", "release_year").show(20, truncate=False)
```

author	release_year
G. A. HENTY	1895
Jane Austen	NULL
Jane Austen	1811
G. A. Henty	1728
JOHN MORLEY	1905
HENRY JAMES	1854
H. Rider Haggard	NULL
THOMAS HARDY	1923
H. RIDER HAGGARD	1912
A. CONAN DOYLE	1905
EDWARD STRATEMEYER	1902
RALPH WALDO EMERSON	1867
Aldous Huxley	1914
ARTHUR CONAN DOYLE	1918
EDMUND BURKE	1886
JACOB ABBOTT	1857
Thomas Carlyle	1417
Bret Harte	1852
Richard Brinsley Sheridan	1775
W. H. GILBERT	1678

```
>>> influence_df.count()
28
```

```
>>> out_degree.show(5)
```

author1	count
EDWARD STRATEMEYER	3
WILLIAM BLAKE	2
Jerome K. Jerome	2
HAMLIN GARLAND	2
Bret Harte	2

only showing top 5 rows

```
>>> in_degree.show(5)
```

author2	count
JACOB ABBOTT	3
Aldous Huxley	2
WILLIAM BLAKE	2
Jerome K. Jerome	2
A. CONAN DOYLE	2

only showing top 5 rows

1. Discuss how you chose to represent the influence network in Spark (e.g., RDD of tuples, DataFrame). What are the advantages and disadvantages of your chosen representation?

➔ I represented the influence network using a Spark DataFrame with columns such as (*author_1*, *author_2*, *influence_score*).

Advantages:

- Optimized query execution using Spark's Catalyst optimizer.
- Easy to use SQL operations for filtering, grouping, and aggregation.
- Better performance for large-scale structured data.

Disadvantages:

- Less flexible than RDDs for complex custom transformations.
- Slight overhead due to schema enforcement.

2. How does the choice of the time window (X) affect the structure of the influence network? What are the limitations of this simplified definition of influence?

➔ The time window (X years) determines whether one author can influence another based on publication years.

- **Small X:** Fewer edges, sparse network, stricter influence definition.
- **Large X:** More edges, denser network, more potential influence links.

Thus, the time window directly affects network density and connectivity.

3. How well would your approach scale to a much larger dataset with millions of books and authors? What optimizations could you consider?

➔ The approach may face challenges due to:

- Large joins between authors.
- Increased memory and computation cost.

Possible Optimizations:

- Use efficient partitioning and indexing.
- Apply broadcast joins for smaller datasets.
- Use filtering to reduce unnecessary comparisons.
- Use graph processing tools like GraphFrames.
- Apply approximate similarity methods instead of full pairwise comparisons.